

Finite State Machines in Game AI: How Automata Theory Powers Enemy Behaviour in Video Games

Sheikh Muhammad Ibrahim Muhammad Ismail Rana Saad Mughal

*Department of Computer Science
Beaconhouse National University*

Lahore, Pakistan

Theory of Automata — 6th Semester, BSCS

Supervised by: Sheeza Batool

May 2026

Abstract—This paper explores how Finite State Machines (FSMs), a core concept from Automata Theory, are used to control enemy behaviour in video games. We explain what FSMs are, how they map directly to the formal definitions studied in class, and how game developers use them to make non-player characters (NPCs) feel intelligent and realistic. We use a simple guard character as a running example and walk through how FSM states and transitions represent that character’s behaviour. The paper also discusses the advantages and limitations of FSMs in game development.

Index Terms—Finite State Machines, Automata Theory, Game AI, Non-Player Characters, Transition Function, NPC Behaviour, State-Driven Design

I. INTRODUCTION

When you play a video game and an enemy starts chasing you the moment it spots you, that behaviour is not random. It follows a set of clear rules: the enemy checks what is happening around it, decides which action to take, and switches between actions based on game events. This kind of logic—switching between actions based on inputs—is exactly what Automata Theory is about.

In a standard Theory of Automata course, students study Finite State Machines (FSMs). An FSM is a mathematical model that has a fixed set of states, a set of inputs, and rules that define how to move from one state to another. These same machines are widely used in game development to control how enemy characters behave.

This paper explains the connection between FSMs and game AI. We show how the formal definition of an FSM matches the way a game developer would program an enemy guard, and why this approach is both practical and effective. The remainder of the paper is organised as follows: Section II reviews the formal definition of an FSM; Section III presents the application of FSMs to game AI; Section IV surveys real-world examples; Section V discusses limitations; Section VI connects the application to course concepts; and Section VII concludes the paper.

II. BACKGROUND: FINITE STATE MACHINES

A Finite State Machine is formally defined as a 5-tuple [3]:

$$M = (Q, \Sigma, \delta, q_0, F) \quad (1)$$

where each component is defined as follows:

- Q — a finite set of *states* (all possible conditions the machine can be in);
- Σ — a finite *input alphabet* (the symbols or events that can cause a change);
- $\delta : Q \times \Sigma \rightarrow Q$ — the *transition function* (given a current state and an input, it returns the next state);
- $q_0 \in Q$ — the *start state* (the state in which the machine begins);
- $F \subseteq Q$ — the set of *accept states* (optional in the game-AI context).

In simple terms, an FSM always knows which state it is in. When something happens (an input arrives), it consults the transition function and moves to the correct next state. It cannot be in two states simultaneously, and it always reacts in a fully predictable, deterministic manner.

III. APPLICATION: FSMs IN GAME AI

A. The Core Idea

Game developers use FSMs to control Non-Player Characters (NPCs)—characters that the computer controls. Each behaviour of an NPC (patrolling, chasing, attacking) becomes a *state*. Events in the game (seeing a player, losing sight of a player, taking damage) become *inputs*. The transition function tells the NPC when to switch from one behaviour to another [1], [2].

This is a direct real-world use of the FSM model: the game designer is essentially building a finite automaton, just described in game terms instead of mathematical notation.

TABLE I
GUARD NPC STATES MAPPED TO FSM COMPONENTS

State	What the Guard Does	FSM Equiv.
Patrol	Walks along a fixed path, looking for the player	q_0 (start)
Chase	Runs toward the player to close the distance	q_1
Attack	Strikes the player when close enough	q_2

B. Case Study: The Guard NPC

Consider a guard enemy in a video game. Its behaviour can be described using a simple FSM with three states, summarised in Table I.

The transitions (inputs) between these states are:

- *See Player* $\Rightarrow$ Patrol $\rightarrow$ Chase
- *Close Enough* $\Rightarrow$ Chase $\rightarrow$ Attack
- *Player Out of Range* $\Rightarrow$ Chase $\rightarrow$ Patrol
- *Player Defeated* $\Rightarrow$ Attack $\rightarrow$ Patrol

In formal notation, the transition function δ is:

$$\delta(\text{Patrol}, \text{"See Player"}) = \text{Chase} \quad (2)$$

$$\delta(\text{Chase}, \text{"Close Enough"}) = \text{Attack} \quad (3)$$

$$\delta(\text{Chase}, \text{"Player Out of Range"}) = \text{Patrol} \quad (4)$$

$$\delta(\text{Attack}, \text{"Player Defeated"}) = \text{Patrol} \quad (5)$$

This is exactly the same structure as the FSMs studied in an Automata Theory course—just with game-world names instead of abstract symbols.

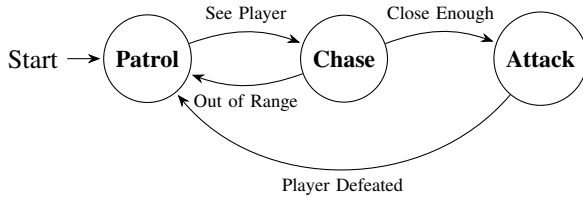


Fig. 1. State-transition diagram of the Guard NPC FSM.

C. Why FSMs Work Well for Game AI

FSMs are popular in game development for several reasons [1], [5]:

- **Simple to understand.** Every behaviour is clearly defined. There is no ambiguity about what the enemy will do in any situation.
- **Easy to design.** A game designer can draw the FSM as a diagram with circles (states) and arrows (transitions) before writing any code.
- **Predictable.** Because the transitions are fixed rules, the NPC always behaves the same way given the same inputs, which makes testing and debugging straightforward.

- **Lightweight.** FSMs use very little memory and processing power, which is important when a game has hundreds of NPCs running simultaneously.

IV. REAL-WORLD EXAMPLES

FSMs are used in many well-known commercial games. Table II summarises four notable examples that span four decades of game development [4].

TABLE II
FSM USAGE IN COMMERCIAL VIDEO GAMES

Game (Year)	FSM Behaviour
Pac-Man (1980)	Each ghost (Blinky, Pinky, Inky, Clyde) cycles through <i>Chase</i> , <i>Scatter</i> , and <i>Frightened</i> states. Eating a power pellet triggers a global transition to <i>Frightened</i> .
DOOM (1993)	Enemy monsters switch between <i>Idle</i> , <i>Alert</i> , and <i>Attack</i> states depending on whether they can see or hear the player.
The Elder Scrolls	Guards and townspeople use FSMs to alternate between daily routines (sleeping, working, wandering) and combat states when attacked.
Halo	Enemy soldiers coordinate between <i>Patrol</i> , <i>Take Cover</i> , <i>Attack</i> , and <i>Retreat</i> states based on the combat situation.

In all these cases the developers implemented the same formal model—the Finite State Machine—that Mealy, Moore, and other computer scientists defined in the 1950s.

V. LIMITATIONS OF FSMs IN GAME AI

Despite their strengths, FSMs exhibit several weaknesses as games grow more complex [1], [2], [5]:

- 1) **State explosion.** If an NPC has many different behaviours (patrol, chase, attack, hide, call for help, retreat, heal, etc.), the number of states and transitions can become enormous and unmanageable.
- 2) **Inflexibility.** FSMs follow fixed rules. An NPC cannot learn or adapt its behaviour based on the player's past actions.
- 3) **Difficult to extend.** Adding a new behaviour to an existing FSM often requires modifying many existing transitions, increasing the risk of introducing bugs.

To mitigate these shortcomings, practitioners have extended the basic FSM model. *Hierarchical FSMs* (HFSMs) group related states into sub-machines, reducing the complexity visible at any single level. *Behaviour Trees* and *Goal-Oriented Action Planning* (GOAP) offer greater flexibility through modular, composable structures. Importantly, these advanced systems are typically built on top of—or inspired by—FSM concepts, confirming that the basic FSM model remains foundational to game AI [4].

VI. CONNECTION TO AUTOMATA THEORY

The game-AI application described in this paper maps directly onto the formal constructs studied in a Theory of

Automata course. Table III presents this correspondence explicitly.

TABLE III
MAPPING AUTOMATA THEORY CONCEPTS TO GAME AI

Course Concept	Game-AI Equivalent
Finite set of states (Q)	The NPC's possible behaviours (Patrol, Chase, Attack)
Alphabet / Input set (Σ)	Game events (See Player, Close Enough, Player Defeated)
Transition function (δ)	The rules deciding when to switch behaviour
Start state (q_0)	The NPC's default behaviour at the beginning of the game
Deterministic transitions	Each game event always leads to exactly one new behaviour state
Accept states (F)	Optional; may represent terminal or goal states in the game

Studying this application demonstrates that the abstract mathematical models from Automata Theory are not merely theoretical exercises: they are actively employed in real software that millions of people interact with every day.

VII. CONCLUSION

Finite State Machines are one of the most practical applications of Automata Theory. Game developers have employed this mathematical model for over four decades to create NPC behaviour that feels intelligent and reactive, without requiring sophisticated artificial intelligence.

The connection between the formal FSM definition $M = (Q, \Sigma, \delta, q_0, F)$ and a game's enemy logic is direct and unambiguous: states are behaviours, inputs are game events, and the transition function encodes the game's decision logic. This makes FSMs a canonical example of how theoretical computer science becomes the foundation for everyday software.

Even as more advanced AI techniques emerge, FSMs remain a fundamental tool in game development because of their simplicity, predictability, and computational efficiency. Understanding FSMs through this application provides a deeper appreciation for the material covered in Theory of Automata and underscores the real-world relevance of formal models in computer science.

REFERENCES

- [1] I. Millington and J. Funge, *Artificial Intelligence for Games*, 2nd ed. Morgan Kaufmann, 2009, ch. 3: Decision Making.
- [2] M. Buckland, *Programming Game AI by Example*. Wordware Publishing, 2004, ch. 2: State-Driven Agent Design.
- [3] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Addison-Wesley, 2006.
- [4] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*. Springer, 2018. [Online]. Available: <http://gameaibook.org>
- [5] A. J. Champandard, *AI Game Development: Synthetic Creatures with Learning and Reactive Behaviors*. New Riders, 2003, ch. 4: Finite-State Machine Architectures.